Bridging the Cost Barrier: A 3D-Printed ROS 2 Platform for RoboCup SSL Benchmarking

Research Proposal

School of Computer Science & Applied Mathematics
University of the Witwatersrand

Sashin Samlal
2459211

Supervised by: Branden Ingram

June 28, 2026



Declaration

I, Sashin Samlal hereby declare the contents of this research document to be my own work. This document is submitted for degree purposes in Computer Science at the University of the Witwatersrand. This work has not been submitted to any other university, or for any other degree.
Date: June 28, 2026

Contents

Preface
Declaration i
Table of Contents ii

1 Introduction **1**
1.1 Problem Statement and Research Questions 2

2 Background **3**
2.1 Design Implications of Technical Constraints 3
2.1.1 Spatial Constraints and Mechanical Packaging 4
2.1.2 Temporal Constraints and Control Theory 4
2.1.3 Information Constraints and Distributed Systems 5

3 Related Work **6**
3.0.1 Locomotion and Drive Mechanisms 6
3.0.2 Evolution of System Architecture and Middleware 7
3.0.3 Accessibility through Cost Optimization 7
3.0.4 Bridging the Gap: Simulation vs. Physical Validation 8

4 Methodology **9**
4.1 Hardware Architecture and Component Selection 9
4.2 Distributed Software Stack and Communication 10
4.3 Control Loop Architecture and Processing Partition 11
4.3.1 Hardware Integration and Sensor Fusion 11
4.3.2 Custom micro-ROS Network Configuration 12
4.3.3 Laptop-Side LTV-MPC Development and Linearization 12
4.3.4 Low-Level Kinematic Tracking and PID Execution 12
4.4 Benchmarking Scenarios and Performance Metrics 13
4.4.1 Benchmarking Scenarios 13
4.4.2 Performance Metrics 14
4.4.3 Experimental Control and Protocol Standardization 15

5 Workplan **16**

References **18**

A 3D CAD Models of the Robot Chassis **19**

B Microcontroller Specification **20**

Chapter 1

Introduction

The barrier to entry for the RoboCup Small Size League (SSL) is primarily defined by the prohibitive cost of specialized robotic hardware and the inherent complexity of high-level algorithm implementation. For instance, a research team must purchase specialized omnidirectional wheels and high-torque brushless motors, costing thousands of dollars, to build a single competitive unit. These limitations often exclude researchers and institutions with limited funding from contributing to the advancement of multi-agent systems and competitive robotics.

In a South African context, where the top 10% of the population captures approximately 70% of the national income [Czajka and Gethin 2026], this creates a severe digital divide that hinders the potential growth of the robotics field. Because robotics education is highly resource-intensive and relies on expensive hardware, software, and stable internet infrastructure under-resourced rural public schools are frequently excluded from these advancements [Mapheto 2023]. Consequently, this financial barrier restricts the broader societal impact of robotics innovation, confining the development of vital 21st-century technological skills primarily to well-resourced private institutions and a privileged economic minority.

A critical solution to this cost barrier lies in the development of a standardized, competition-capable robotic platform that utilizes additive manufacturing and open-source middleware to simplify hardware-software integration. Such a design provides an accessible entry point, fostering wider participation in the SSL and stimulating broader interest in autonomous systems.

Historically, low-cost approaches have relied on a "hobbyist" paradigm, utilizing off-the-shelf boards and structural components sourced from general hardware suppliers, such as sheet metal. While various open-source middleware solutions exist to facilitate software standardization, integrating disparate hobbyist components remains a significant challenge due to proprietary communication protocols and manufacturer-specific constraints. Beyond component integration, traditional manufacturing methods like welding or metalwork require specialized skills to meet the strict dimensional tolerances mandated by league regulations.

However, the rapid advancement and cost-effectiveness of 3D printing technology now allow for the production of standardized, modular, and easily reproducible structural components. By integrating this hardware with the Robot Operating System (ROS) middleware, it is possible to create a standardized interface that abstracts

low-level control and enables seamless transition between simulation and physical deployment.

The establishment of this unified hardware-software framework further permits the implementation of standardized benchmarking suites, providing a rigorous environment for the performance evaluation of diverse robotic components and autonomous strategies.

1.1 Problem Statement and Research Questions

Although additive manufacturing and open-source middleware lower the barrier to entry for robotic research, it remains unproven whether a non-holonomic, low-cost platform can satisfy the strict operational constraints of a competitive environment. Standard RoboCup SSL Division B regulations require high accelerations and rapid deceleration capabilities that are historically achieved using expensive, specialized omnidirectional drive mechanisms [Avidano *et al.* 2023]. This study addresses the core problem of verifying whether an accessible, 3D-printed differential-drive robot can perform within these strict motion and safety bounds.

The primary research question asks whether a low-cost, 3D-printed differential-drive platform running under a dual-loop control architecture can satisfy the velocity, deceleration, and path-following requirements defined by the RoboCup SSL Division B technical standards. To evaluate this primary question, two secondary inquiries are established. First, we investigate the quantitative impact of non-holonomic kinematics on trajectory tracking accuracy across distinct path classes, focusing on the trade-off between mechanical simplicity and trajectory deviation. Second, we examine the extent to which the scheduling jitter and communication latency of the Robot Operating System 2 (ROS 2) and micro-ROS message bridge degrade control loop stability.

Based on these inquiries, we propose two hypotheses. First, we hypothesize that the physical platform can sustain a maximum velocity of 1.5 m/s and decelerate to a complete stop within the 2.0 s limit mandated by the league. Second, we hypothesize that the network latency of the micro-ROS Best-Effort transport protocol remains below the 16.7 ms vision frame interval, preventing instability in the closed-loop controller.

Chapter 2

Background

The RoboCup SSL represents a highly constrained environment designed to promote research in fast-paced multi-robot systems. To understand the benchmarking of a custom low-cost platform against standard solutions, it is necessary to establish the regulatory and physical parameters of the league. Table 2.1 summarizes the core technical standards mandated by the league.

Table 2.1: RoboCup SSL Division B Technical Standards [RoboCup Small Size League Technical Committee 2026].

Parameter	Regulation	Design Impact
Robot Volume	$180\,\mathrm{mm} \times 150\,\mathrm{mm}$ height	Constrains motor size and wheel geometry.
Ball Velocity	Max $6.5\,\mathrm{m/second}$	Defines limits for solenoid/pneumatic kicker systems.
Ball Interaction	$< 20\%$ coverage	Prevents ball hoarding; requires precise dribbler geometry.
Halt Latency	< 2.0s to zero velocity	Mandates high-frequency braking control.
Stop Speed	Max $1.5\,\mathrm{m/second}$	Requires real-time speed regulation in the control stack.
Vision	Overhead SSL-Vision	Necessitates dot-pattern ID and distributed processing.

2.1 Design Implications of Technical Constraints

These regulations are not merely boundary conditions; they directly influence the scientific and engineering trade-offs required to build a competition-capable platform.

2.1.1 Spatial Constraints and Mechanical Packaging

The strict 180mm $\times$ 150mm cylindrical volume creates a complex "packing problem" for the robotic chassis. In competitive, high-cost units, this volume is typically maximized using four-wheel omnidirectional drives to achieve holonomic movement. However, implementing such a drive requires precise geometric placement of four independent motors and wheels within the circular convex hull, significantly increasing mechanical complexity and decreasing the space available for batteries and low-level electronics [Avidano *et al.* 2023].

By contrast, a differential-drive system simplifies this spatial constraint by utilizing only two independent drive wheels. While this introduces non-holonomic constraints, which require the robot to rotate before translating, it allows for a more stable and modular internal layout. This study benchmarks whether such a simplified mechanical design, optimized for 3D printing, can still meet the maneuverability requirements of the league while remaining financially accessible.

To model the motion of this differential-drive configuration, we establish the kinematic equations relating the wheel velocities to the robot velocities. Let r represent the wheel radius, and let L denote the distance between the two drive wheels. The state of the robot in the global coordinate frame is defined by the vector $\mathbf{x} = [x, y, \theta]^T$, where (x, y) represents the midpoint of the wheel axle and θ is the heading angle. The linear velocity v and angular velocity ω in the robot's local frame are calculated from the right wheel angular velocity $\dot{\phi}_R$ and left wheel angular velocity $\dot{\phi}_L$:

$$v = \frac{r}{2}(\dot{\phi}_R + \dot{\phi}_L) \tag{2.1}$$

$$\omega = \frac{r}{L}(\dot{\phi}_R - \dot{\phi}_L) \tag{2.2}$$

The global state update equations are:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \tag{2.3}$$

2.1.2 Temporal Constraints and Control Theory

The league's safety protocols, specifically the "Halt" and "Stop" commands, impose rigorous temporal constraints on the control architecture. The requirement to reach zero velocity within two seconds of a "Halt" signal necessitates a control stack capable of high-frequency sampling and rapid deceleration profiles. In addition to halting quickly, maintaining a maximum speed of 1.5 m/s during "Stop" phases requires precise velocity estimation. These constraints motivate a hierarchical, dual-loop control architecture. A high-level Linear Time-Varying Model Predictive Control (LTV-MPC) loop runs on the central computer to compute dynamically feasible trajectories under velocity and acceleration constraints, while a low-level Proportional-Integral-Derivative (PID) loop operates on the microcontroller to regulate the wheel angular velocities, ensuring that the physical platform adheres to the safety standards defined by the referee box (refbox) [Pacheco and Luo 2015].

The velocity controller regulates the wheel angular velocities. The error $e(t)$ between the target wheel velocity $\dot{\phi}_{target}(t)$ and measured velocity $\dot{\phi}_{measured}(t)$ is:

$$e(t) = \dot{\phi}_{target}(t) - \dot{\phi}_{measured}(t) \tag{2.4}$$

The control signal $u(t)$ is calculated using a continuous PID control law:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{2.5}$$

where K_p, K_i, and K_d represent the proportional, integral, and derivative gains respectively. For digital implementation on the microcontroller, this control law is discretized at a sampling interval Δt:

$$u_k = K_p e_k + K_i \Delta t \sum_{j=0}^{k} e_j + \frac{K_d}{\Delta t}(e_k - e_{k-1}) \tag{2.6}$$

where k represents the current time step.

2.1.3 Information Constraints and Distributed Systems

The prohibition of human intervention and the reliance on the centralized SSL-Vision system motivate a distributed "software glue" architecture. Since all positional data is generated by overhead cameras and distributed via wireless packets, the robot cannot rely on local on-board vision.

This creates a dependency on network latency and external state estimation. To address this, the proposed architecture utilizes a Single Board Computer (SBC) running ROS to bridge the gap between centralized vision commands and local motor execution [Smit 2011; Joseph 2018]. This distributed intelligence allows the platform to handle global strategy while maintaining local control stability, an essential requirement for high-speed autonomous play.

The centralized vision server transmits tracking data over a network via User Datagram Protocol (UDP) multicast packets. The payload of each packet is serialized using Google Protocol Buffers, containing a nested data structure defined as:

```
message SSL_DetectionFrame {
  required double t_capture = 1;
  required double t_sent = 2;
  required uint32 camera_id = 3;
  repeated SSL_DetectionBall balls = 4;
  repeated SSL_DetectionRobot robots_yellow = 5;
  repeated SSL_DetectionRobot robots_blue = 6;
}
```

where each robot packet contains its team ID, coordinates (x, y), and orientation θ.

Chapter 3

Related Work

The development of robotic platforms for competitive environments like the RoboCup SSL has evolved through distinct shifts in locomotion strategy, software architecture, and manufacturing philosophy. This section explores these transitions to motivate the design of a low-cost, 3D-printed benchmarking platform.

3.0.1 Locomotion and Drive Mechanisms

The high-speed, dynamic nature of robotic soccer has traditionally necessitated the use of complex holonomic drives to allow for simultaneous translation and rotation. While platforms designed for the SSL overwhelmingly utilize omnidirectional drive mechanisms, their dominance is justified by the essential requirement to decouple heading from velocity vectors. This decoupling allows a robot to maintain its orientation toward the ball or a target while translating in any direction, a capability that is essential for high-frequency ball interception and defensive "side-shuffling."

Despite their tactical superiority, omnidirectional systems impose a significant "complexity tax." Implementing a four-wheel omni-drive requires the geometric synchronization of four independent motors, which increases the probability of mechanical failure and reduces the internal volume available for power-dense components. Recent literature, such as the *First Order* Team Description Paper (TDP) [Liang *et al.* 2025], highlights that the transition to a compact holonomic base often requires a complete mechanical redesign to manage these spatial trade-offs.

By contrast, the differential-drive approach represents a strategic sacrifice of agility for the sake of mechanical durability and cost-efficiency. The non-holonomic penalty represents a key performance bottleneck; differential robots must perform a "rotate-then-translate" maneuver, which introduces a measurable latency in response to rapid changes in ball trajectory. Tactically, this limitation makes differential platforms less effective in goal-tending scenarios where lateral movement is paramount.

Quantitatively, benchmarking studies demonstrate that differential drive robots can experience an increase in "Time-to-Target" metrics and path planning complexity compared to holonomic peers [Chavan and Minase 2015]. However, for a low-cost 3D-printed platform, the differential drive simplifies the control stack and reduces manufacturing barriers. The proposed project utilizes the differential approach, following the success of the *MICROMVP* platform [Yu *et al.* 2017], to determine if the loss in agility

is a justifiable trade-off for a benchmarking-focused research platform.

3.0.2 Evolution of System Architecture and Middleware

The development of the RoboCup SSL has been marked by a fundamental shift from centralized "dumb" agents to distributed, intelligent architectures. Early systems relied on a single off-field computer to process vision data and transmit low-level actuator duty cycles directly to robots. However, the increasing complexity of multi-robot coordination and the need to reduce dependency on high-bandwidth wireless links motivated the adoption of on-board SBCs capable of local state estimation and motor control [Smit 2011].

The transition to distributed intelligence necessitated a standardized "software glue" to manage inter-process communication. While early research utilized the Player/Stage middleware, ROS emerged as the dominant standard due to its modular "computational graph" and extensive ecosystem of visualization and diagnostic tools [Joseph 2018]. ROS simplified the integration of disparate hardware by providing a common messaging interface, allowing researchers to focus on high-level algorithm development rather than low-level driver implementation.

Despite its success, the evolution from ROS1 to ROS2 represents a significant technical milestone for the SSL environment. ROS1 utilized a centralized "ROS Master" and a TCP-based transport layer, which introduced a single point of failure and was ill-suited for the unreliable wireless networks of a competition floor. In contrast, ROS2 adopts a decentralized architecture based on the Data Distribution Service (DDS). By eliminating the Master and introducing configurable Quality of Service (QoS) policies (such as "Best Effort" and "Transient Local"), ROS2 provides the reliability and multi-robot discovery capabilities required for dynamic agent interaction [Macenski *et al.* 2022].

However, the adoption of ROS introduces significant performance trade-offs, particularly regarding real-time determinism. Standard ROS running on a non-preemptive Linux kernel is inherently non-deterministic; the standard Completely Fair Scheduler (CFS) prioritizes overall throughput over meeting strict millisecond deadlines [Kronauer *et al.* 2021]. This introduces "jitter" into control loops that can degrade the stability of high-speed platforms. In addition to scheduling jitter, the communication overhead of message serialization and deserialization, the process of transforming data structures into binary buffers for the DDS, can introduce latencies that impact the 60Hz feedback cycles provided by SSL-Vision [Gutiérrez *et al.* 2018]. This study analyzes whether these architectural overheads can be mitigated through zero-copy intra-process communication or if they necessitate the use of PREEMPT_RT kernel patches to bridge the "reality gap" in software timing.

3.0.3 Accessibility through Cost Optimization

The high barrier to entry in robotics is often defined by manufacturing complexity. To overcome this, recent literature highlights a move away from expensive machined metal toward additive manufacturing. Teams increasingly utilize 3D-printed plastics to reduce costs and increase design flexibility for complex components [Liang *et al.* 2025].

This accessibility is further enhanced by combining 3D printing with off-the-shelf electronics. The *MICROMVP* platform demonstrates that a highly affordable micro-scale testbed is achievable through this combination [Yu *et al.* 2017]. This layout is extended by Joseph [2018], who provide a framework for bridging high-level ROS nodes running on an SBC directly to low-level, inexpensive microcontrollers, an approach central to this project's hardware design.

A critical distinction exists between low-cost educational platforms and competition-grade RoboCup SSL robots. Standard educational platforms, such as the TurtleBot or the NVIDIA JetBot, prioritize simplicity, safety, and general-purpose research interfaces over high-speed dynamical capabilities [Joseph 2018]. These educational platforms operate at low velocities, typically under 0.5 m/s, and utilize simple differential-drive kinematics that are ill-suited for the rapid acceleration and dynamic maneuvering required on a competition field. Conversely, competition-grade RoboCup SSL robots are highly optimized, holonomic systems running at speeds exceeding 3.0 m/s with sub-millisecond control latencies [Liang *et al.* 2025]. These competition-grade platforms utilize specialized omnidirectional wheels and high-torque brushless motors, which significantly increase structural complexity and financial cost [Avidano *et al.* 2023]. The proposed project addresses this gap by investigating whether an accessible, 3D-printed differential-drive platform can satisfy the strict motion and safety requirements of the competitive SSL environment, thereby offering a viable middle ground for under-resourced research institutions.

3.0.4 Bridging the Gap: Simulation vs. Physical Validation

Validation of multi-agent algorithms often occurs in virtual environments to bypass the costs of physical testing. For the RoboCup Middle Size League, Yao *et al.* [2015] develop a simulation system based on ROS and Gazebo, utilizing a custom model plugin to mathematically simulate locomotion.

While such simulations simplify robots into basic cylindrical collision elements to preserve computational speed, they require the injection of Gaussian noise to mimic real-world sensor inaccuracies [Yao *et al.* 2015]. This "reality gap" motivates the necessity of the current project: by building a physical benchmarking platform, we can quantitatively evaluate how algorithms perform against real-world friction, battery dynamics, and discretization delays that simplified virtual models often fail to capture within the constraints defined by the official league rules [RoboCup Small Size League Technical Committee 2026].

Chapter 4

Methodology

4.1 Hardware Architecture and Component Selection

The physical chassis of the robotic platform adopts a cylindrical geometry to facilitate rotation within confined spatial envelopes, drawing structural design principles from the multi-vehicle platform presented by Yu *et al.* [2017]. Integrating three distinct cutouts, the base plate features two arch-shaped apertures accommodating the primary differential wheels alongside a third rectangular opening for a passive caster wheel that maintains vertical stability. Structural fabrication utilizes three-dimensional (3D) printing techniques to support rapid prototyping and low-cost replication. Visual representations of the three-dimensional CAD chassis models are detailed in Appendix A, showing the top view (Figure A.1a), front view (Figure A.1b), and side view (Figure A.1c). An open-top design characterizes the casing, which employs thickened mounting structures to secure the microcontroller. Convective airflow promoted by this open configuration cools internal electronics and provides direct physical access to the motor terminals. Soldered connections are applied to all electrical interfaces to maximize conductivity and minimize the probability of short-circuits during high-acceleration maneuvers.

The physical dimensions of the platform are constrained by the RoboCup SSL regulations, which mandate that all participating robots must fit within a cylinder of 180 mm diameter and 150 mm height [RoboCup Small Size League Technical Committee 2026]. Remaining comfortably within these limits, the proposed design utilizes a footprint of 100 mm diameter and 100 mm height. Symmetrical cylinders minimize rotational inertia and simplify the 3D-printing process relative to asymmetric geometries. Internal space accommodates a Raspberry Pi Pico microcontroller board, which measures 51 mm by 21 mm. A visual layout of the microcontroller is detailed in Appendix B (Figure B.1), which is retrieved from the official product specification sheet [Raspberry Pi Foundation 2026]. Running micro-ROS, this board executes a specialized client library enabling microcontrollers to communicate directly with a central ROS environment [Macenski *et al.* 2022]. Low unit cost and compatibility with hardware extension boards, such as Wi-Fi and camera modules, support modular hardware scaling. Although an onboard camera is excluded from the initial prototype to minimize cost, the physical chassis design reserves mounting space to accommodate future computer vision integration.

Locomotion is driven by two spur gear DC motors equipped with integrated magnetic quadrature encoders, representing a mechatronic configuration modeled after the motor drive systems that Salem [2013] analyzes. The spur gear motors are selected due to their affordability, general availability, and integrated internal gearheads that increase output torque at low rotational velocities. The quadrature encoders measure the rotational speed of each wheel to provide feedback for the local velocity controller. To track the dynamic motion of the platform, the chassis houses an MPU6050 three-axis Inertial Measurement Unit (IMU) containing both an accelerometer and a gyroscope. This sensor communicates with the microcontroller via an Inter-Integrated Circuit (I^2C) interface and fits within a compact footprint, facilitating mounting near the center of mass of the robot to collect accurate chassis-level acceleration and orientation data.

4.2 Distributed Software Stack and Communication

The control software is structured around ROS 2 and its microcontroller-compatible extension, micro-ROS, establishing a distributed client-agent communication network [Macenski *et al.* 2022]. Primary navigation and path-planning algorithms run on a centralized laptop serving as the central host running ROS 2 nodes. Velocity commands calculated by this central host are published over the network. Subscribing to these topics, the micro-ROS client running on the Raspberry Pi Pico receives the trajectory data and processes it for execution. Closing the feedback loop, the microcontroller publishes local odometry and IMU telemetry from the quadrature encoders and the MPU6050 sensor back to the ROS 2 host. Sensor streams received by the host are integrated into the path planning and localization nodes to dynamically update subsequent trajectory commands.

Position and orientation tracking are achieved via an external overhead camera system. Overhead detection relies on a unique visual marker, or sigil, affixed to the top of the robot chassis, which allows the tracking software to resolve the coordinate position and heading angle of the vehicle in real time. This localization framework follows the multi-vehicle tracking pipeline that Yu *et al.* [2017] describe. Coordinate streams are processed by the central laptop to determine the position of the robot relative to the SSL playing field, serving as the input for path-generation algorithms. Figure 4.1 illustrates this distributed system architecture and the data pipeline mapping raw camera coordinates and sensor feedback to wheel velocities.

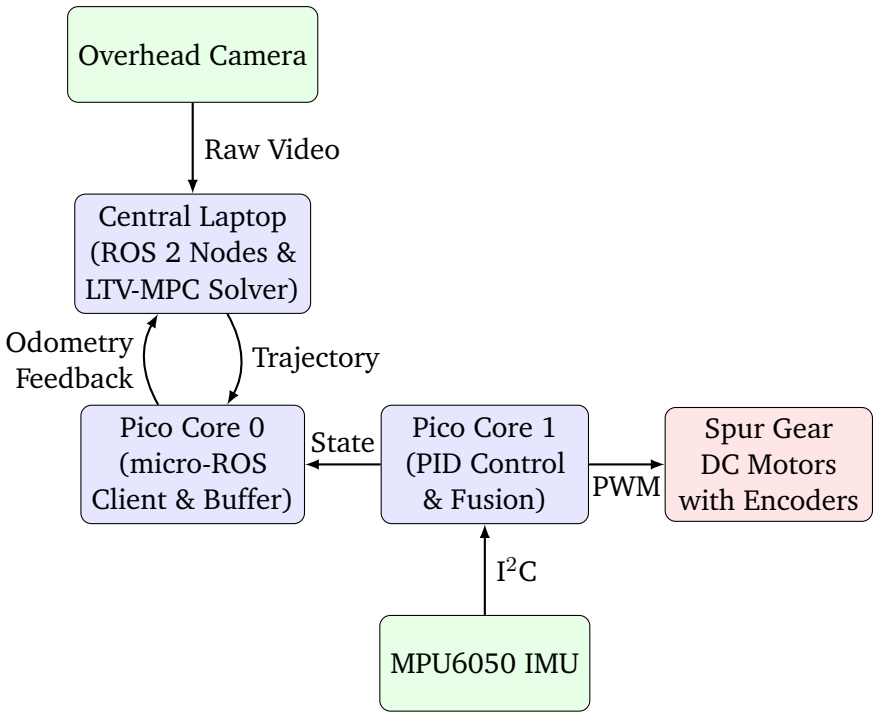


Figure 4.1: Distributed system architecture and data pipeline.

4.3 Control Loop Architecture and Processing Partition

To guarantee real-time determinism and prevent communication overhead from interfering with time-critical motor control, the software stack is partitioned across the dual-core architecture of the Raspberry Pi Pico.

4.3.1 Hardware Integration and Sensor Fusion

To prevent measurement errors induced by mechanical rotation, the GY-521 breakout board housing the MPU6050 IMU is mounted precisely at the geometric center of the drive-wheel axle line. This physical alignment eliminates false centripetal acceleration readings:

$$a_y = \omega_z^2 \cdot r_{\text{offset}} \tag{4.1}$$

where $r_{\text{offset}} = 0$ is the radial distance from the axle center to the sensor. Sensor reads are executed on Core 1 of the microcontroller using a non-blocking I^{2}C polling routine with Direct Memory Access (DMA) clocked at 400 kHz in Fast Mode. This isolates the communication link, ensuring that sensor data collection does not cause the microcontroller to drop time-critical wheel encoder interrupts.

A local fixed-point Complementary Filter runs on Core 1 to fuse high-frequency gyroscope heading data with the gravity vector from the accelerometer:

$$\theta_{\text{fused},k} = \alpha(\theta_{\text{fused},k-1} + \omega_{z,k}\Delta t) + (1-\alpha)\theta_{\text{accel},k} \tag{4.2}$$

where $\theta_{\text{fused},k}$ is the current fused heading, $\omega_{z,k}$ is the gyroscope yaw rate, $\theta_{\text{accel},k}$ is the accelerometer-derived heading, Δt is the sampling interval, and $\alpha = 0.98$ is the filter

coefficient. This algorithm mitigates gyroscope thermal drift before the fused heading is passed to Core 0 via the hardware FIFO mailbox.

4.3.2 Custom micro-ROS Network Configuration

To prevent packet fragmentation over the wireless link, a dedicated ROS 2 message profile, `TrajectoryProfile.msg`, is defined to transmit compact representations of the target path from the central laptop to the robot. The individual elements of this profile are defined as:

```
# TrajectoryStep.msg
float32 v_target
float32 omega_target
uint32 delta_ms
```

The micro-ROS publisher for IMU and odometry data on the Pico is configured to use a Best-Effort Quality of Service (QoS) transport policy. This transport policy prevents dropped Wi-Fi packets from triggering retransmission cycles, isolating the control loop from network jitter. To secure microsecond-accurate timestamp alignment for latency logging, the Pico runs the `rmw_uros_epoch_nanos()` time service client on Core 0 to continuously sync its internal clock with the laptop's master ROS 2 clock.

4.3.3 Laptop-Side LTV-MPC Development and Linearization

The high-level path-following controller is implemented as an Error-Model Linear Time-Varying Model Predictive Control (LTV-MPC) node on the laptop using C++ and the high-performance 'OsqpEigen' solver [Pacheco and Luo 2015; Stellato *et al.* 2020]. The non-holonomic differential kinematics:

$$\mathbf{x}_{k+1} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k \tag{4.3}$$

are linearized around the reference path at every camera tracking frame update, where state vector $\mathbf{x} = [x, y, \theta]^T$ and control input vector $\mathbf{u} = [v, \omega]^T$. The solver minimizes a quadratic cost function J over a finite horizon. The physical RoboCup SSL Division B constraints are mapped directly into the optimization matrix:

$$-1.5 \text{ m/s} \leq v_k \leq 1.5 \text{ m/s} \tag{4.4}$$

and dynamic acceleration limits:

$$-\alpha_{\text{max}} \leq a_k \leq \alpha_{\text{max}} \tag{4.5}$$

are set to satisfy the 2.0-second safety halting rule (t_{stop}).

4.3.4 Low-Level Kinematic Tracking and PID Execution

Core 0 of the Pico receives the incoming array of 5 to 10 trajectory steps from the laptop and caches them sequentially in a trajectory horizon buffer. Core 1 extracts the active

trajectory target, calculates the target wheel velocities via inverse kinematics, and feeds the values directly into the PID velocity loop. Core 1 runs at 500 Hz, interpolating intermediate steps between the laptop's lower-frequency MPC updates (20 Hz $-$ 50 Hz) to guarantee smooth motor transitions.

The wheel velocity controller regulates the individual motor speeds. The error $e_i(t)$ between the target wheel velocity $\dot{\phi}_{i,\text{target}}(t)$ and measured velocity $\dot{\phi}_{i,\text{measured}}(t)$ is defined as:

$$e_i(t) = \dot{\phi}_{i,\text{target}}(t) - \dot{\phi}_{i,\text{measured}}(t) \tag{4.6}$$

where $i \in \{\text{left}, \text{right}\}$ denotes the motor. The control signal $u_i(t)$ sent to the motor drivers is:

$$u_i(t) = K_p e_i(t) + K_i \int_0^t e_i(\tau) d\tau + K_d \frac{de_i(t)}{dt} \tag{4.7}$$

where K_p, K_i, and K_d represent the gains tuned to prevent motor overshoot.

4.4 Benchmarking Scenarios and Performance Metrics

To evaluate the performance of the 3D-printed differential-drive robot, the platform is benchmarked against two baseline systems: a laboratory TurtleBot and an NVIDIA JetBot [NVIDIA Corporation 2026].

4.4.1 Benchmarking Scenarios

The benchmarking process is divided into three distinct physical scenarios:

1. *Straight-Line Sprint and Halt*: The robot drives forward in a straight line for 5 meters at maximum acceleration. Upon crossing the 5-meter mark, a halt command is broadcast, and the robot must stop.

2. *Trajectory Tracking*: The robot tracks three classes of reference paths to evaluate steering performance under non-holonomic constraints:

    - *Straight Line*: Evaluates drift and lateral error due to wheel misalignment.
    - *Circular Arc*: Evaluates steady-state turning and constant-radius trajectory tracking.
    - *M-Shape*: Evaluates transient response, sharp cornering capability, and rapid deceleration/acceleration phases. This path is representative of defensive blocking and evading maneuvers in RoboCup matches.

3. *Step-Response Latency*: The robot is commanded to start moving from a standstill. The time delay between command transmission and physical wheel rotation is recorded.

4.4.2 Performance Metrics

We define seven quantitative metrics to evaluate performance across these scenarios:

1. *Maximum Velocity* (v_{max}): The maximum velocity achieved during the straight-line sprint:
$$v_{\text{max}} = \max \frac{\Delta d}{\Delta t} \tag{4.8}$$
RoboCup SSL rules restrict this to 1.5 m/s [RoboCup Small Size League Technical Committee 2026].

2. *Stopping Time* (t_{stop}): The time elapsed between the halt command timestamp t_{halt} and the moment the robot's velocity reaches zero t_{zero}:
$$t_{\text{stop}} = t_{\text{zero}} - t_{\text{halt}} \tag{4.9}$$
RoboCup SSL rules require $t_{\text{stop}} \leq 2.0$ s [RoboCup Small Size League Technical Committee 2026].

3. *Stopping Distance* (d_{stop}): The physical Euclidean displacement traveled after a halt command is issued:
$$d_{\text{stop}} = \|\mathbf{p}(t_{\text{halt}}) - \mathbf{p}(t_{\text{zero}})\| \tag{4.10}$$
where $\mathbf{p}(t)$ is the two-dimensional position vector.

4. *Path Error* (E_{path}): The root-mean-square error (RMSE) between target and actual coordinates:
$$E_{\text{path}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}[(x_k - \hat{x}_k)^2 + (y_k - \hat{y}_k)^2]} \tag{4.11}$$
where (x_k, y_k) is the target coordinate and $(\hat{x}_k, \hat{y}_k)$ is the tracked position.

5. *Command Latency* (τ_{latency}): The communication delay:
$$\tau_{\text{latency}} = t_{\text{motion}} - t_{\text{command}} \tag{4.12}$$
where t_{command} is the laptop's command transmission timestamp and t_{motion} is the timestamp when physical movement is registered by the IMU.

6. *Turning Radius* (R_{turn}): The minimum turning circle radius that the robot can draw.

7. *System Consistency* (σ_x): The standard deviation of each metric over 10 test iterations:
$$\sigma_x = \sqrt{\frac{1}{n-1}\sum_{j=1}^{n}(x_j - \bar{x})^2} \tag{4.13}$$
where $n = 10$, x_j is the recorded metric value for iteration j, and $\bar{x}$ is the mean.

4.4.3 Experimental Control and Protocol Standardization

To ensure reproducibility, the following calibration and experimental protocols are established:

- *Camera Homography Calibration*: Before executing trials, a homography transformation matrix is calculated using ground control points to map overhead camera pixels directly to physical playing field coordinates in millimeters.
- *Clock Synchronization*: The micro-ROS time synchronization client on Core 0 synchronizes the Pico's internal clock with the laptop's master clock at startup and at regular intervals, ensuring sub-millisecond coordination.
- *Data Logging*: All published topics (including control commands, encoder ticks, IMU readings, and camera coordinates) are recorded locally on the laptop in ROS bag files during every trial.
- *Battery State Monitoring*: Tests are only executed when the robot's battery voltage is above a threshold of 11.1 V (for a 3S LiPo battery) to ensure consistent motor torque.
- *Surface Consistency*: All trials are conducted on the same flat carpet surface to maintain a constant coefficient of friction.

Chapter 5

Workplan

The proposed honors research project is structured across five sequential phases over a seventeen-week period from June 30 to October 25, 2026. This timeline provides dedicated periods for low-level dual-core firmware design, laptop-side optimization solver integration, and statistical evaluation against multiple comparative baselines. Table 5.1 details the start dates, end dates, and target deliverables for each project milestone.

Table 5.1: Proposed Schedule of Work (June 30 – October 25)

Task	Description	Start Date	End Date
1	Chassis fabrication: 3D printing of chassis, base plate, motor brackets, and casing.	30-06-2026	10-07-2026
2	Mechanical assembly: Mount Raspberry Pi Pico, MPU6050, motors, caster, and solder interfaces.	11-07-2026	20-07-2026
3	Firmware development: Implement dual-core task partition, non-blocking I^2C, and sensor fusion.	21-07-2026	10-08-2026
4	Network configuration: Establish custom message interface and Best-Effort QoS on Pico.	11-08-2026	25-08-2026
5	Vision calibration: Calibrate camera homography and tracking sigils on field.	26-08-2026	05-09-2026
6	Laptop-side solver: Implement C++ LTV-MPC node with OSQP solver and constraints.	06-09-2026	25-09-2026
7	Controller integration: Integrate high-level MPC with low-level PID speed loops.	26-09-2026	05-10-2026
8	Benchmarking trials: Execute performance scenarios on platform, TurtleBot, and JetBot.	06-10-2026	15-10-2026
9	Compilation: Analyze statistical results and write final thesis report.	16-10-2026	25-10-2026

References

[Avidano *et al.* 2023] C. Avidano, S. Barnette, M. Barulic, L. Medrano, J. Neiger, R. Osawa, E. Peterson, J. Spall IV, J. Spalten, W. Stuckey, and M. Woodward. *The A-Team Technical Description Paper 2023*. Team description paper, RoboCup Small Size League, 2023.

[Chavan and Minase 2015] AV Chavan and JL Minase. Design of a differential drive mobile robot platform for use in constrained environments. *International Journal of Innovations in Engineering Research and Technology*, 2(6):1–10, 2015.

[Czajka and Gethin 2026] Léo Czajka and Amory Gethin. *Thirty years of inequality and redistribution in post-Apartheid South Africa*, March 2026. Published 26.03.26 under Institutions & Political Economy.

[Gutiérrez *et al.* 2018] Carlos San Vicente Gutiérrez, Lander Usategui San Juan, Irati Zamalloa Ugarte, and Víctor Mayoral Vilches. Towards a distributed and real-time framework for robots: Evaluation of ROS 2.0 communications for real-time robotic applications. *arXiv preprint arXiv:1809.02595*, 2018.

[Joseph 2018] Lentin Joseph. *Robot operating system (ros) for absolute beginners*, volume 2018. Springer, 2018.

[Kronauer *et al.* 2021] Tobias Kronauer, Joshwa Pohlmann, Maximilian Matthé, Till Smejkal, and Gerhard Fettweis. Latency analysis of ROS 2 multi-node systems. *arXiv preprint arXiv:2101.02074*, 2021.

[Liang *et al.* 2025] Martin Liang, Joel Ng, Fred Huang, Louis Yong, Pi Rong Koh, Hamish Starling, and Luke Moran. *First Order Robotics Team Description Paper Small Size League of RoboCup 2025*. RoboCup Small Size League, 2025.

[Macenski *et al.* 2022] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.

[Mapheto 2023] Doctor Mapheto. *Teachers' and learners' acceptance of the use of robotics in the Intermediate Phase*. Magister educationis dissertation, University of Pretoria, August 2023.

[NVIDIA Corporation 2026] NVIDIA Corporation. *NVIDIA JetBot Developer Guide*. https://jetbot.org, 2026. Retrieved June 6, 2026.

[Pacheco and Luo 2015] Lluis Pacheco and Ningsu Luo. Testing PID and MPC performance for mobile robot local path-following. *International Journal of Advanced Robotic Systems*, 12(155):1–13, 2015.

[Raspberry Pi Foundation 2026] Raspberry Pi Foundation. *Raspberry Pi Pico*. https://www.raspberrypi.com/products/raspberry-pi-pico/, 2026. Retrieved May 25, 2026.

[RoboCup Small Size League Technical Committee 2026] RoboCup Small Size League Technical Committee. *Rules of the RoboCup Small Size League*, February 2026. Last updated 2026-02-25.

[Salem 2013] Farhan A. Salem. Dynamic and kinematic models and control for differential drive mobile robots. *International Journal of Current Engineering and Technology*, 3(2):253–263, 2013.

[Smit 2011] Albert Smit. *Development of a robot for RoboCup Small Size League, utilizing a distributed control architecture for a multi-robot system development platform*. PhD thesis, Stellenbosch: Stellenbosch University, 2011.

[Stellato *et al.* 2020] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.

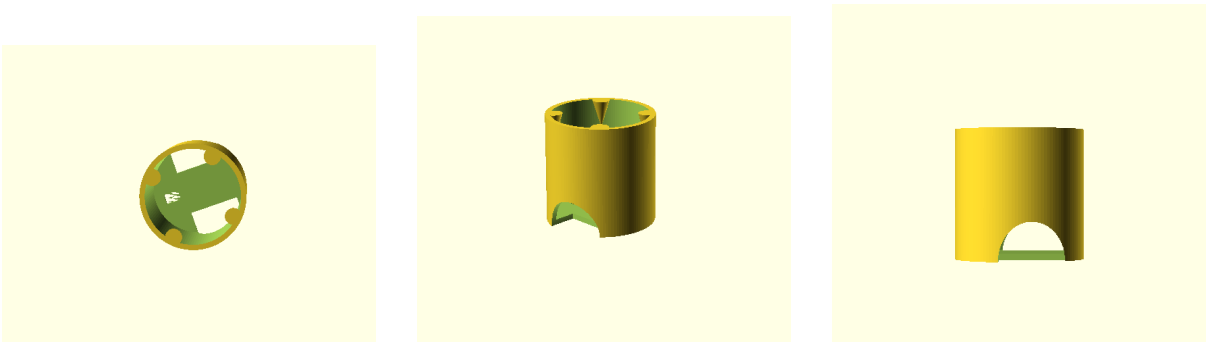
[Yao *et al.* 2015] Weijia Yao, Wei Dai, Junhao Xiao, Huimin Lu, and Zhiqiang Zheng. A simulation system based on ros and gazebo for robocup middle size league. In *2015 IEEE international conference on robotics and biomimetics (ROBIO)*, pages 54–59. IEEE, 2015.

[Yu *et al.* 2017] Jingjin Yu, Shuai D Han, Wei N Tang, and Daniela Rus. A portable, 3d-printing enabled multi-vehicle platform for robotics research and education. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1475–1480. IEEE, 2017.

Appendix A

3D CAD Models of the Robot Chassis

This appendix contains the three-dimensional computer-aided design (CAD) model renderings of the robot chassis body. Figure A.1 shows the top view, front view, and side view of the design.



(a) Top view. (b) Front view. (c) Side view.

Figure A.1: Three-dimensional CAD models of the robot chassis body.

Appendix B

Microcontroller Specification

This appendix presents the technical layout of the microcontroller. Figure B.1 illustrates the Raspberry Pi Pico board layout retrieved from the official documentation [Raspberry Pi Foundation 2026].

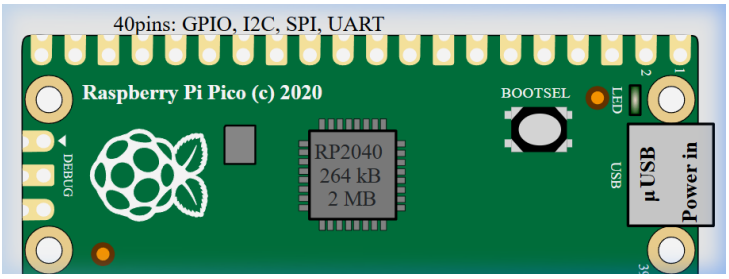


Figure B.1: Raspberry Pi Pico board layout [Raspberry Pi Foundation 2026].

Wits University Faculty of Science post-graduate student AI declaration

I understand that the use of generative AI tools (such as ChatGPT or similar) without explicitly declaring such use constitutes a form of plagiarism and is classified by Wits University as academic misconduct.

I declare that in the course of conducting the research towards my degree or in the preparation of this thesis/dissertation/research report (select one by marking with an X):

I **did not** make use of generative AI tools ☐

I **did** make use of generative AI tools for the following (tick all that apply):

1. Idea Generation (research problem/design, hypothesis) ☒
2. Sourcing Related Work (summarising, identifying sources) ☒
3. Methods and Experiment Design (experiment setup, model tuning) ☐
4. Data Analysis (presentation, coding, interpretation) ☐
5. Theoretical Development (theorem proving, conceptual analysis) ☐
6. Code Development (generating algorithms, writing scripts) ☐
7. Presentation (rendering graphics, formatting) ☐
8. Editing (grammar, readability) ☒
9. Writing (text generation, document structuring) ☒
10. Citation Formatting (structuring, organising) ☐

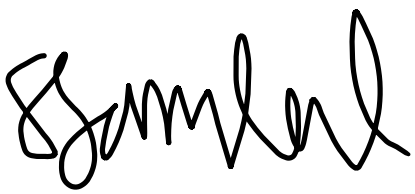
If other uses were involved, please specify below:

Generative AI tool used (list all)	**Used for?**

If generative AI tools were used as an integral part of the experimental design or in the direct execution of my research, I confirm that details of this use are clearly outlined in the relevant experimental/methodology chapters of my thesis/dissertation/research report.

Student number: 2459211

Candidate signature:



Date: 09-04-2026